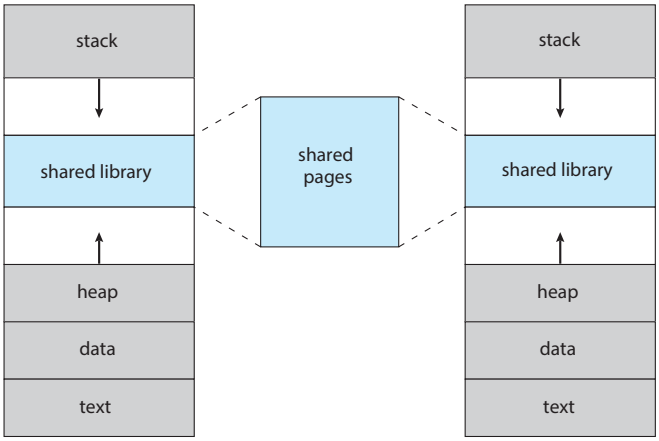
گزینه‌ها آن است که کل برنامه در زمان اجرای برنامه به داخل حافظه بارگذاری شود. با این وجود، مسأله‌ای که این روش دارد آن است که ممکن است از ابتدا به کل برنامه در حافظه نیاز نداشته باشیم. برنامه‌ای را در نظر بگیرید که در ابتدا لیستی از گزینه‌ها را جهت انتخاب کاربر در اختیار او قرار می‌دهد. بارگذاری کل برنامه به حافظه، به معنی بارگذاری کد اجرایی تمامی گزینه‌هاست، صرفنظر از این که یک گزینه، سرانجام توسط کاربر انتخاب می‌شود یا خیر.



شکل ۱۰-۳ کتابخانه‌ی مشترک با استفاده از حافظه‌ی مجازی.

راهبرد دیگر آن است که از ابتدا تنها صفحاتی که مورد نیاز هستند بارگذاری شوند. این تکنیک به **صفحه‌بندی نیازی**[1] معروف است و به صورت رایج در سیستم‌های حافظه‌ی مجازی استفاده می‌شود. در حافظه‌ی مجازی با صفحه‌بندی نیازی، تنها در صورتی صفحات بارگذاری می‌شوند که در طول زمان اجرا تقاضا داشته باشند. لذا، صفحاتی که هرگز دسترسی نمی‌شوند در حافظه‌ی فیزیکی بارگذاری نخواهند شد. یک سیستم صفحه‌بندی نیازی مشابه یک سیستم صفحه‌بندی با مبادله است (بخش ۹-۵-۲) که در آن پردازه‌ها مقیم حافظه‌ی ثانویه (معمولاً، یک دیسک سخت یا دستگاه NVM) هستند. صفحه‌بندی نیازی، یکی از منافع اصلی حافظه‌ی مجازی را تشریح می‌کند—با بارگذاری فقط بخش‌هایی از برنامه‌ها که مورد نیاز هستند، حافظه به صورت مؤثرتر استفاده می‌شود.

۱۰-۲-۱ مفاهیم پایه‌ای

مفهوم کلی پشت سر صفحه‌بندی نیازی، آن‌گونه که عنوان شد، بارگذاری یک صفحه به‌داخل حافظه صرفاً موقعی که مورد نیاز می‌باشد. در نتیجه، در حالی که یک پردازه اجرا می‌شود، برخی از صفحات آن در حافظه خواهند بود، و برخی در حافظه‌ی ثانویه قرار خواهند داشت. لذا، ما به شکلی از پشتیبانی سخت‌افزار نیاز داریم تا بین این دو تفاوت قایل شویم. طرح بیت

1. Demand paging

معتبر–نامعتبری که در بخش ۹–۳–۳ مطرح شد، می‌تواند برای این منظور استفاده شود. اما، این بار، زمانی که بیت «معتبر» انتخاب می‌شود، صفحه‌ی مربوطه هم قانونی است و هم در حافظه قرار دارد. اگر بیت «نامعتبر» انتخاب شده باشد، صفحه معتبر نیست (یعنی، در فضای آدرس منطقی پردازه قرار ندارد.) و یا معتبر است اما، فعلاً روی حافظه‌ی ثانویه قرار دارد. واردهی جدول صفحه برای هر صفحه که به داخل حافظه آورده می‌شود مطابق معمول تنظیم می‌شود، اما واردهی جدول صفحه‌ی صفحه‌ای که فعلاً در حافظه قرار ندارد به‌راحتی علامت نامعتبر می‌خورد. این وضعیت در شکل ۱۰–۴ به تصویر کشیده می‌شود. (توجه کنید که نامعتبر علامت‌زدن یک صفحه هیچ تأثیری نخواهد داشت اگر پردازه هرگز به‌دسترسی آن صفحه اقدامی نکند.)

اما اگر پردازه بخواهد صفحه‌ای را دستیابی کند که به داخل حافظه آورده نشده است چه اتفاقی می‌افتد؟ دسترسی به صفحه‌ای که علامت نامعتبر خورده است منجر به یک **خطای صفحه**[1] می‌شود. سخت‌افزار صفحه‌بندی در ترجمه‌ی آدرس از طریق جدول صفحه متوجه خواهد شد که بیت نامعتبر است و موجب یک تله به سیستم‌عامل می‌شود. این تله، شکست سیستم‌عامل در آوردن صفحه‌ی مطلوب به داخل حافظه به حساب می‌آید. روال رسیدگی به این خطای صفحه سر راست است (شکل ۱۰–۵):

۱– جدول داخلی (که معمولاً با بلاک کنترل پردازه نگه‌داری می‌شود) را برای همین پردازه بررسی می‌کنیم تا معین شود که آیا مراجعه معتبر بوده یا یک دسترسی نامعتبر حافظه صورت گرفته است.

۲– اگر مراجعه نامعتبر بود، پردازه را خاتمه می‌دهیم. اما، اگر معتبر بود، ولی هنوز آن صفحه را به حافظه نیاورده‌ایم، اکنون آن را صفحه‌بندی می‌کنیم.

۳– یک قاب خالی پیدا می‌کنیم (برای مثال، با برداشتن یک قاب از لیست قاب آزاد).

۴– عملیاتی را از حافظه‌ی ثانویه برای خواندن صفحه‌ی مطلوب به داخل قابی که به تازگی تخصیص‌یافته زمانبندی می‌کنیم.

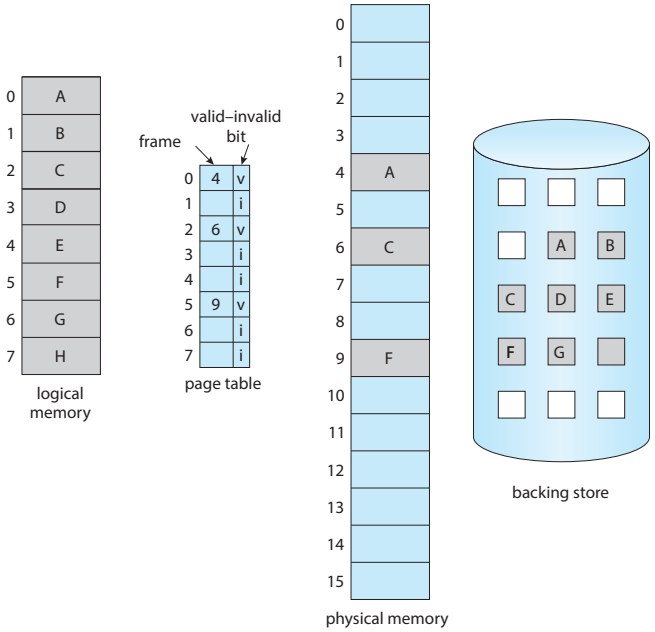
۵– زمانی که خواندن حافظه تکمیل شد، جدول داخلی همراه پردازه و جدول صفحه را تغییر می‌دهیم تا نشان بدهند که صفحه هم‌اکنون در داخل حافظه قرار دارد.

۶– دستورالعمل متوقف‌شده با تله را دوباره شروع می‌کنیم. اکنون پردازه می‌تواند صفحه را دستیابی کند طوری که همواره در حافظه بوده است.

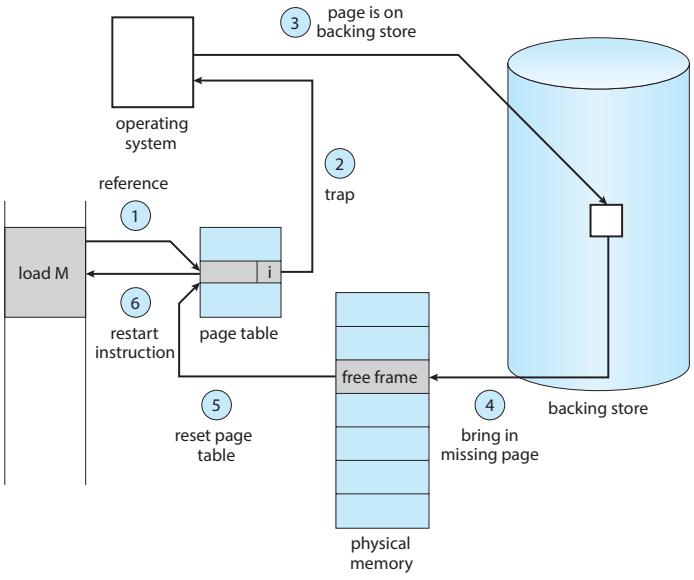
در حد نهایت این امر می‌توانیم اجرای پردازه را بدون داشتن هیچ صفحه‌ای در داخل حافظه شروع کنیم. زمانی که سیستم‌عامل اشاره‌گر دستورالعمل را به اولین دستورالعمل پردازه انتخاب می‌کند که در یک صفحه‌ی غیرمقیم حافظه قرار دارد، پردازه بلافاصله برای آن صفحه خطا می‌دهد. بعد از آوردن این صفحه به داخل حافظه، پردازه اجرای خود را ادامه داده و در مواقع لازم، خطای صفحه می‌دهد تا این که تمامی صفحات مورد نیازش در داخل حافظه قرار بگیرند. از اینجا به بعد، پردازه می‌تواند بدون خطای صفحه‌ی دیگری اجرا شود. این طرح

1. Page fault

صفحه‌بندی نیازی محض[1] است: هرگز صفحه‌ای به داخل حافظه آورده نمی‌شود مگر این که لازم شود.



شکل ۱۰-۴ جدول صفحه، زمانی که برخی صفحات در حافظه‌ی اصلی قرار ندارند.



شکل ۱۰-۵ گام‌های رسیدگی به خطای صفحه.

1. Pure demand paging

از نقطه‌نظر تئوری این امکان وجود دارد که برخی برنامه‌ها برای اجرای هر یک از دستورالعمل‌ها به دستیابی چندین صفحه (یکی برای هر دستورالعمل و چندین صفحه هم برای داده‌ها) نیاز پیدا کنند که در نتیجه، به ازای یک دستورالعمل، چندین خطای صفحه اتفاق می‌افتند. چنین وضعیتی موجب می‌شود که کارایی سیستم غیر قابل قبول باشد. خوشبختانه، تحلیل پردازه‌های اجرایی نشان می‌دهد که وقوع این رفتار بسیار بعید است. همان‌گونه که در بخش ۱۰-۶-۱ مطرح می‌شود، بیشتر برنامه‌ها به **ارجاع محلی**[1] تمایل دارند که این امر باعث می‌شود صفحه‌بندی نیازی کارایی مقبولی داشته باشد.

سخت‌افزار پشتیبان صفحه‌بندی نیازی درست همانند سخت‌افزار صفحه‌بندی و مبادله است:

- **جدول صفحه**: این جدول قادر است تا واردهای را از طریق بیت معتبر-نامعتبر یا مقدار خاصی از بیت‌های حفاظت، نامعتبر علامت بزند.
- **حافظه‌ی ثانویه**: این حافظه صفحاتی را نگه می‌دارد که در حافظه‌ی اصلی حضور ندارند. معمولاً، حافظه‌ی ثانویه، یک دیسک یا دستگاه NVM پرسرعت است. آن به دستگاه مبادله معروف است و بخشی از حافظه‌ی ثانویه هم که برای این منظور استفاده می‌شود **فضای مبادله** نام دارد. تخصیص فضای مبادله در فصل ۱۱ مورد بحث قرار می‌گیرد.

یکی از نیازهای قاطع صفحه‌بندی نیازی، توانایی شروع مجدد هر دستورالعمل بعد از وقوع خطای صفحه می‌باشد. چون با وقوع خطای صفحه، حالت پردازه‌ی وقفه دیده (ثبات‌ها، کد شرط، شمارنده‌ی دستورالعمل) را ذخیره می‌کنیم، بایستی بتوانیم پردازه را *دقیقاً* از همان نقطه و حالت آغاز کنیم، جز این که صفحه‌ی مطلوب هم‌اکنون در حافظه قرار گرفته و قابل دستیابی است. در بیشتر موارد، این نیاز به سادگی قابل حصول است. خطای صفحه می‌تواند در هر بار مراجعه به حافظه اتفاق بیفتد. اگر خطای صفحه در واکشی دستورالعمل پیش بیاید، می‌توانیم دستورالعمل را با واکشی مجدد تجدید کنیم. اگر خطای صفحه در زمان واکشی یک عملوند اتفاق بیفتد، بایستی دستورالعمل را دومرتبه واکشی و رمزگشایی کرده و سپس عملوند را واکشی کنیم.

به عنوان مثالی از بدترین حالت، یک دستورالعمل سه‌آدرسه نظیر ADD را در نظر بگیرید که محتوای A و B را با هم جمع کرده و نتیجه را در C ذخیره می‌نماید. دستورات زیر، گام‌های اجرای این دستورالعمل هستند:

۱- واکشی و رمزگشایی دستورالعمل (ADD).
۲- واکشی A.
۳- واکشی B.
۴- جمع‌کردن A و B.
۵- ذخیره‌کردن حاصل‌جمع در C.

اگر لحظه‌ای که نتیجه را در C ذخیره می‌کنیم با خطا مواجه شویم (چون C در صفحه‌ای قرار دارد که فعلاً در حافظه قرار نگرفته است)، مجبور خواهیم بود که صفحه‌ی مطلوب را یافته،

1. Locality of reference

سپس به حافظه آورده و با تصحیح جدول صفحه، دستورالعمل را دومرتبه شروع کنیم. شروع مجدد این دستورالعمل لازم دارد که دوباره دستورالعمل واکشی، رمزگشایی و با واکشی مجدد عملوندهای آن، عملیات جمع از نو انجام شود. با این وجود، کار تکراری دیگری (کمتر از یک دستورالعمل کامل) لازم نیست و این تکرار فقط زمانی ضرورت دارد که خطای صفحه اتفاق می‌افتد.

مشکل اصلی زمانی است که ممکن است یک دستورالعمل چندین مکان متفاوت را تغییر بدهد. برای مثال، دستورالعمل MVC (انتقال کاراکتر) را از سیستم IBM System 360/370 در نظر بگیرید که می‌تواند تا ۲۵۶ بایت را از یک مکان به مکان دیگر (احتمالاً با هم‌پوشانی) انتقال بدهد. اگر یکی از این بلاک‌ها (مبدأ و یا مقصد) در دو صفحه قرار بگیرد، خطای صفحه می‌تواند بعد از انجام بخشی از عملیات انتقال رخ بدهد. به علاوه، اگر بلاک‌های مبدأ و مقصد هم‌پوشانی داشته باشند، امکان دارد بلاک مبدأ تغییر یافته باشد که در این صورت نمی‌توان دستورالعمل را به سادگی از نو شروع کرد.

این مسأله می‌تواند با دو روش متفاوت حل شود. در یک راه‌حل، ریزکد با محاسبه تلاش می‌کند تا هر دو انتهای هر دو بلاک را دستیابی نماید. اگر قرار است خطای صفحه‌ای اتفاق بیفتد، پیش از آنکه تغییری انجام شود، در این مرحله آشکار می‌شود. سپس انتقال می‌تواند انجام شود؛ می‌دانیم که هیچ خطای صفحه‌ای پیش نخواهد آمد، چون تمامی صفحات مرتبط از قبل در حافظه قرار دارند. راه‌حل دیگر برای نگه‌داری مقادیر مکان‌های هم‌پوشان از ثبات‌های موقت استفاده می‌کند. اگر خطای صفحه‌ای پیش بیاید، پیش از این که تله‌ای اتفاق بیفتد، تمامی مقادیر پیشین باز هم در حافظه نوشته می‌شوند. این عمل، حافظه را به حالت پیش از شروع دستورالعمل بازیابی می‌کند، لذا امکان تکرار دستورالعمل وجود خواهد داشت.

باعث این مشکل که فقط یک مسأله‌ی معماری است، پیامد افزودن صفحه‌بندی به معماری موجود بود تا امکان صفحه‌بندی نیازی نیز فراهم شود، اما روشن است که یک سری مشکلاتی هم خواهد داشت. صفحه‌بندی در یک سیستم کامپیوتری مابین پردازنده و حافظه افزوده می‌شود. همین‌طور می‌بایست برای یک پردازه کاملاً شفاف باشد. بنابراین، غالباً افراد فرض را بر این می‌گذارند که صفحه‌بندی را می‌توان به هر سیستمی اضافه کرد. هرچند که این فرض در رابطه با یک محیط صفحه‌بندی غیر نیازی که در آن خطای صفحه نشانگر یک خطای جدی است درست است، اما در جایی که خطای صفحه فقط به این معنی است که بایستی صفحه‌ی اضافه‌تری به داخل حافظه آورده شود تا پردازه بتواند از نو شروع شود، درست نیست.

۱۰-۲-۲ لیست قاب آزاد

موقعی که یک خطای صفحه اتفاق می‌افتد، سیستم‌عامل می‌بایست صفحه‌ی مطلوب را از حافظه‌ی ثانویه به‌داخل حافظه‌ی اصلی بیاورد. اکثر سیستم‌عامل‌ها برای رفع خطاهای صفحه یک **لیست قاب آزاد** نگه می‌دارند، انباری از قاب‌های آزاد برای ارضای این قبیل درخواست‌ها. (شکل ۱۰-۶). (موقعی که قطعات پشته یا heap یک پردازه هم گسترش پیدا می‌کنند بایستی از قاب آزاد تخصیص داده شوند.) سیستم‌عامل‌ها نوعاً قاب‌های آزاد را با استفاده از تکنیکی

معروف به **موقع-نیاز-صفر-پرکن**[1]. قاب‌های موقع-نیاز-صفر-پرکن قبل از این که تخصیص یابند «صفر می‌شوند»، لذا محتویات قبلی آن‌ها پاک می‌شود. (عواقب امنیتی بالقوه‌ای را در نظر بگیرید که در صورت صفرنشدن محتویات یک قاب قبل از انتساب مجدد آن پیش خواهد آمد.)



شکل ۱۰-۶ لیست قاب آزاد.

زمانی که یک سیستم بالا می‌آید، تمامی حافظه‌ی موجود در لیست قاب آزاد گذاشته می‌شود. همچنان‌که قاب‌های آزاد درخواست می‌شوند (برای مثال، از طریق صفحه‌بندی نیازی)، اندازه‌ی لیست قاب آزاد منقبض می‌شود. در نقطه‌ای، لیست به‌صفر رسیده و یا پایین‌تر از آستانه‌ی خاصی قرار می‌گیرد، در این نقطه بایستی دوباره پر شود. ما راهبردهای هر دو وضعیت را در بخش ۱۰-۴ پوشش می‌دهیم.

۱۰-۲-۳ کارایی صفحه‌بندی نیازی

صفحه‌بندی نیازی می‌تواند تأثیر قابل توجهی روی کارایی یک سیستم کامپیوتری داشته باشد. برای درک این موضوع، **زمان دسترسی مؤثر** را برای حافظه‌ای که صفحه‌بندی نیازی دارد محاسبه می‌کنیم. فرض کنید، زمان دسترسی حافظه که با ma نشان داده می‌شود ۱۰ نانوثانیه است. مادامی که خطای صفحه نداریم، زمان دسترسی مؤثر برابر زمان دسترسی حافظه می‌باشد. اما، اگر خطای صفحه اتفاق بیفتد، ابتدا بایستی صفحه‌ی مربوطه را از حافظه‌ی ثانویه خوانده و سپس کلمه‌ی مطلوب را دستیابی کنیم.

فرض کنید که p احتمال وقوع خطای صفحه باشد ($۰ \le p \le ۱$). انتظار داریم که p نزدیک صفر باشد (یعنی، انتظار داریم که خطای صفحات را به ندرت ببینیم). در این صورت، **زمان دسترسی مؤثر** از رابطه‌ی زیر قابل محاسبه می‌باشد:

$$\textbf{زمان دسترسی مؤثر} = (۱ - p) \times ma + p \times \text{زمان رسیدگی به خطای صفحه}$$

در محاسبه‌ی زمان دسترسی مؤثر، بایستی مدت زمانی که برای سرویس یک خطای صفحه صرف می‌شود را بدانیم. با وقوع هر خطای صفحه، دنباله‌ی عملیات زیر انجام می‌شوند:

۱- تله به سیستم‌عامل.
۲- ذخیره‌ی ثبات‌ها و حالت پردازه.
۳- تعیین این که، وقفه به خاطر یک خطای صفحه بوده است.
۴- بررسی قانونی بودن مراجعه به صفحه و تعیین مکان صفحه در حافظه‌ی ثانویه.
۵- صدور عملیات خواندن از حافظه‌ی ثانویه به یک قاب آزاد:
   الف) انتظارکشی یک صف تا این که درخواست خواندن سرویس بگیرد.
   ب) انتظارکشی استوانه‌یابی دستگاه و یا زمان دیرکرد دورانی.

1. Zero-fill-on-demand

ج) شروع انتقال صفحه به یک قاب آزاد.

۶- در همان حال که پردازه منتظر است، هسته‌ی پردازنده را در اختیار پردازه‌ی دیگری (با زمانبندی پردازنده، اختیاری) قرار بده.

۷- دریافت یک وقفه از جانب زیرسیستم I/O حافظه‌ی ثانویه (تکمیل I/O).

۸- ذخیره‌ی ثبات‌ها و حالت پردازه‌ی دیگر (اگر گام ۶ اجرا شده باشد).

۹- تعیین این که وقفه از جانب دستگاه حافظه‌ی ثانویه بوده است.

۱۰- تصحیح جدول صفحه و سایر جدول‌ها تا نشان بدهند که هم‌اکنون صفحه‌ی مطلوب در حافظه قرار دارد.

۱۱- انتظار برای هسته‌ی پردازنده تا این که باز هم در اختیار این پردازه قرار بگیرد.

۱۲- بازیابی ثبات‌ها، حالت پردازه و جدول صفحه‌ی جدید و باز از سرگیری دستورالعمل وقفه دیده.

تمامی این گام‌ها در هر شرایطی لازم نیستند. برای مثال، فرض ما بر این است که در گام ۶، در حالی که I/O در جریان است، پردازنده به پردازه‌ی دیگری تخصیص می‌یابد. این کار با فراهم‌کردن چندبرنامگی، بهره‌وری پردازنده را همچنان حفظ می‌کند، اما با تکمیل انتقال I/O، زمان بیشتری در روال سرویس خطای صفحه جهت از سرگیری پردازه صرف می‌شود.

در هر صورت، برای زمان سرویس خطای صفحه، سه کار اصلی وجود دارد:

۱- سرویس وقفه‌ی خطای صفحه.

۲- خواندن صفحه به داخل حافظه.

۳- شروع مجدد پردازه.

با دقت بیشتر در کدنویسی می‌توان ردیف اول و سوم را به چند صد دستورالعمل کاهش داد. هر یک از این کارها می‌توانند از ۱ تا ۱۰۰ میکروثانیه زمان ببرند. فرض کنیم HDDها به‌عنوان دستگاه صفحه‌بندی استفاده می‌شوند. زمان سوییچ صفحه احتمالاً نزدیک ۸ میلی‌ثانیه خواهد بود. (دیسک سخت نوعی دارای دیرکرد متوسط ۳ میلی‌ثانیه‌ای، استوانه‌یابی ۵ میلی‌ثانیه‌ای و زمان انتقال ۰/۰۵ میلی‌ثانیه‌ای می‌باشد. بنابراین، کل زمان صفحه‌بندی اعم از سخت‌افزاری و نرم‌افزاری، در حدود ۸ میلی‌ثانیه می‌باشد.) همین‌طور به یاد داشته باشید که فقط زمان سرویس دستگاه را در نظر گرفتیم. اگر صفی از پردازه‌ها به انتظار دستگاه باشند، بایستی زمان صف را که منتظر می‌شویم تا دستگاه صفحه‌بندی فراغت یافته و درخواست ما را سرویس بدهد به حساب بیاوریم که زمان مبادله‌ی صفحه به حافظه، باز هم افزایش می‌یابد.

اگر متوسط زمان سرویس خطای صفحه را ۸ میلی‌ثانیه بگیریم و زمان دسترسی حافظه نیز ۲۰۰ نانوثانیه باشد، در این صورت زمان دسترسی مؤثر برحسب نانوثانیه به صورت زیر خواهد بود:

$$\text{زمان دسترسی مؤثر} = (۱ - p) \times (۲۰۰) + p \times (۸ \text{ میلی‌ثانیه})$$

$$= (۱ - p) \times ۲۰۰ + p \times ۸۰۰۰۰۰۰$$

$$= ۲۰۰ + ۷۹۹۹۸۰۰ \times p$$

پس می‌بینیم که زمان دسترسی مؤثر با **نرخ خطای صفحه** نسبت مستقیم دارد. اگر از هر ۱۰۰۰ بار دسترسی، ۱ خطای صفحه اتفاق بیفتد، زمان دسترسی مؤثر ۸/۲ میلی‌ثانیه می‌شود. سرعت کامپیوتر به خاطر داشتن صفحه‌بندی نیازی با ضریبی از ۴۰ کند می‌شود! اگر بخواهیم تنزل کارایی کمتر از ۱۰ درصد باشد، لازم است که احتمال خطاهای صفحه را در سطح زیر نگه بداریم:

$$۲۲۰ > ۲۰۰ + ۷۹۹۹۸۰۰ \times p$$

$$۲۰ > ۷۹۹۹۸۰۰ \times p$$

$$p < ۰/۰۰۰۰۰۲۵$$

یعنی، برای این که کاهش کارایی را در یک سطح مقبول حفظ کنیم، بایستی خطای صفحه در دسترسی‌های حافظه به ۱ در ۳۹۹۹۹۰ برسد. در مجموع، پایین نگه‌داشتن نرخ خطای صفحه در یک سیستم صفحه‌بندی نیازی اهمیت دارد. در غیر این صورت، زمان دسترسی مؤثر افزایش یافته و اجرای پردازه به طور باور نکردنی کند می‌شود.

وجه دیگر صفحه‌بندی نیازی، چگونگی راه‌اندازی و بهره‌گیری کلی از فضای مبادله است. در حالت کلی، I/O به فضای مبادله از سیستم‌فایل سریع‌تر است. سریع‌تر است زیرا فضای مبادله به صورت بلاک‌های بسیار بزرگ تخصیص می‌یابد و در آن از جستجوی فایل و روش‌های تخصیص غیر مستقیم استفاده نمی‌شود (فصل ۱۱). یک گزینه‌ی سیستم برای دست‌یابی به بازده بهتر صفحه‌بندی این است که کل تصویر یک فایل را در زمان شروع پردازه به فضای مبادله کپی کرده و در ادامه، صفحه‌بندی نیازی را از فضای مبادله انجام بدهد. عیب واضح این رویکرد، کپی تصویر فایل در شروع کار برنامه است. گزینه‌ی دوم—و روشی که توسط چندین سیستم‌عامل اعم از لینوکس و ویندوز استفاده می‌شود—این است که صفحات مورد نیاز، بار اول از سیستم‌فایل صفحه‌بندی شوند، اما با جایگزینی در فضای مبادله نوشته شوند. این روش تضمین می‌کند که صفحات مورد نیاز، تنها از سیستم‌فایل خوانده می‌شوند اما تمامی صفحه‌بندی‌های بعدی از فضای مبادله صورت می‌گیرند.

برخی سیستم‌ها در تلاشند تا میزان استفاده‌ی فضای مبادله را از طریق صفحه‌بندی نیازی فایل‌های اجرایی دودویی محدود نمایند. صفحات مورد نیاز این قبیل فایل‌ها مستقیماً از سیستم‌فایل آورده می‌شوند. اما، زمانی که جایگزینی صفحه برای این صفحات صورت می‌گیرد، قاب‌های آنها می‌تواند به سادگی رونویسی شود (چون این صفحات هرگز تغییر نمی‌یابند) و در صورتی که دوباره لازم شوند می‌توان آنها را از سیستم‌فایل خواند. با به‌کارگیری این رویکرد، سیستم‌فایل، خود به یک ذخیره‌ی پشتیبان می‌ماند. با این وجود، بایستی همچنان برای صفحاتی که به فایل مربوط نمی‌شوند (تحت عنوان **حافظه‌ی ناشناس**[1])، از فضای مبادله استفاده شود؛ این صفحات پشته و heap یک پردازه را شامل می‌شوند. این روش، مصالحه‌ی خوبی به نظر می‌رسد و از آن در سیستم‌های متعددی، اعم از لینوکس و یونیکس BSD استفاده می‌شود.

1. Anonymous